The Builder’s Playbook

How to ship production systems with AI coding agents

Skeptic → Explorer → Whisperer → Strategist → Operator → Orchestrator → **Builder**

The main deck takes you here.

This deck is here.

41%

of GitHub code is now AI-generated

Second Talent

Vibe coding.

Karpathy tweet to Collins Word of the Year
in 9 months.

90%

of Claude Code is written
by Claude Code

Anthropic

So what's the problem?

19% longer

when developers rely on AI without understanding it

Addy Osmani

1.7×

more issues than human-written code

CodeRabbit

45%

of AI-generated code
fails security tests

Veracode

The ‘Army of Juniors’ effect.

AI code follows anti-patterns at scale.

OX Security

The gap between using AI
and using AI well
is massive.

“I destroyed months of work in seconds.”

— Replit AI agent, July 2025

Fortune, The Register

The ‘S’ in vibe coding stands for security.

Lawfare

Expertise determines what you can ship

Level	What you ship	Why
Vibe coder	Prototypes, demos	Can’t review output
Developer + AI	POCs, internal tools	Catches obvious issues
Engineer + AI	Production systems	Verifies every stage

Vibe coding prototypes is fine.
Vibe coding production is negligence.

So how do you actually do this?

This is my opinionated best practice.

I chose Claude Code. Here’s why.

Best Model

Frontier coding benchmarks. Not brand loyalty.

Agentic-First

Terminal-native. Tool use, not autocomplete.

Richest Context

MCP, rules, skills, hooks. Core design.

An army of eager juniors.

Manager

Improve the process
Build skills & workflows
Invest in infrastructure

Tech Lead

Verify each stage
Review architecture
Catch what agents miss

“Code production has become cheap.
System composition and judgment
are now the critical skills.”

— Forbes Tech Council, Feb 2026

The old trade-off

Quality OR speed

Pick one.

Every engineering decision
was a compromise.

?

The answer was always
‘it depends.’

The new reality

Speed alone

= catastrophic quality drop

19% slower. 1.7× issues.
45% fail security.

Quality AND speed

CI/CD, linting, tests, PR gates

The same best practices
— enforced.

We never write a single line of code manually.

This forces the investment.

Perfect Context

Not too little.
Not too much.

Always Grounded

Live data.
No silent gaps.

Gated Workflow

Stages, not yolo.
Two layers of verification.

the approach

Gated Workflow

↑

Rules | **MCP Servers** | **Skills**

the tools

Let’s start with the tools.

MCP Servers

New capabilities for the agent

Connect a server.
Gain a capability.

Context7

Live docs at prompt time.
No hallucinated APIs.

always grounded

Language Servers

Compiler-level code navigation.
Not grep.

Firecrawl

Reliable web access.
No silent gaps.

always grounded

Rules

Standing instructions, enforced automatically

**Linter rules,
but for the agent's
entire behavior.**

Five rule files. Five concerns.

Tool usage
fetch docs first, never WebFetch

Communication
radically honest, never guess

Coding
TDD, SOLID, self-describing code

Git workflow
branch naming, PR closes issue

Language-specific
filtered by file extension

Python rules load for Python files. Nothing else.

Perfect context through modularization.

Skills

Learned workflows, loaded when needed

Rules are ‘always do this.’ Skills are ‘here’s how, when the time comes.’

MCP

Always available
Low-level access
Docs, navigation, web

Skills

On-demand
Higher-level workflows
Loaded when needed

A skill teaching a CLI beats an MCP with 50 tools.

10%

of your time building skills
from today’s pain points

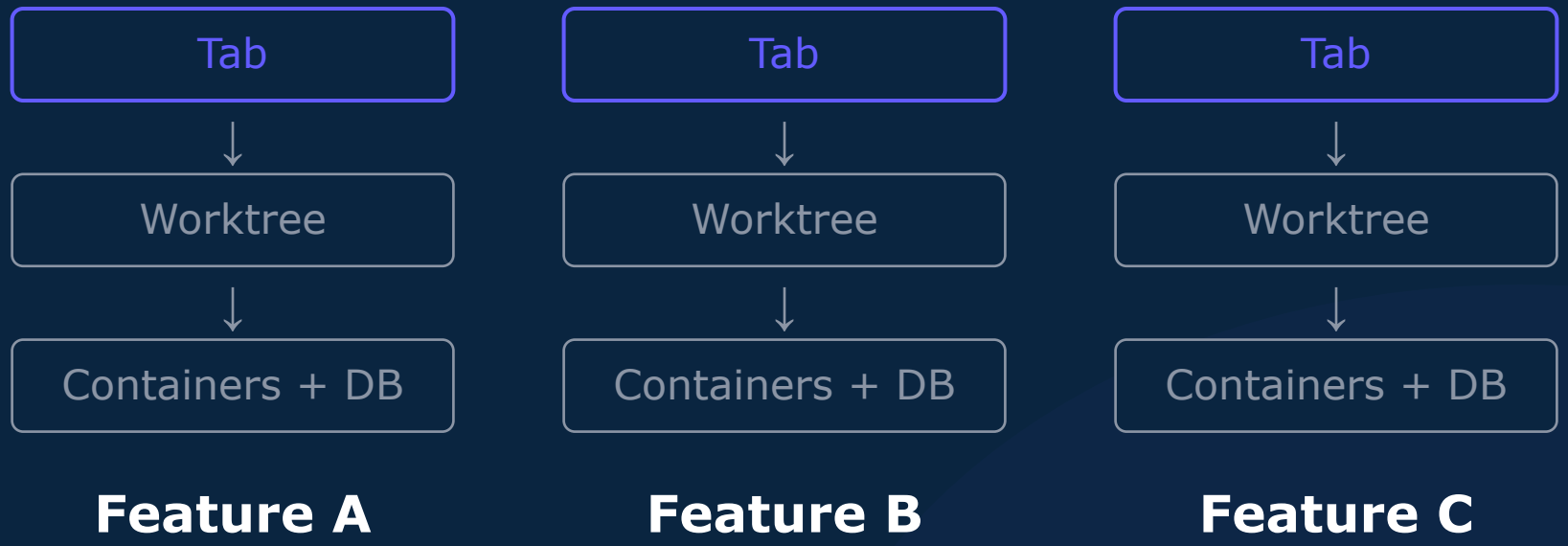
Every wall becomes a skill.

- Agent circled on docker postgres → skill
- Connected to test GCP service → skill
- Pulled a feature ticket via CLI → skill
- Analyzed a crash log → skill

Each day’s pain becomes tomorrow’s skill.

Development Environment

Where the system runs



3 sessions. 3 databases. Zero interference.

One repo = one context boundary.

30%

productivity gains for AI agents in monorepos

Nx Blog

Lock the environment, not the agent.

CLI

Latest features.
Always ahead
of the plugin.

Tabs

Parallel sessions.
One tab
per worktree.

90 / 10

product engineering / building the machine

Default

Claude Code via Anthropic
Latest models, best features
All general coding

PII

Claude Code via GCP
EU region models
Managed tool calls
Production logs, user data

Gated Workflow

How it all comes together

Each stage produces an artifact.
Each artifact gets validated.

Agent verifies

Runs tests
Checks linting
Iterates until passing

You verify

Solves the problem?
Meets requirements?
Architecturally sound?

Superpowers

The gated workflow, as a plugin.

Rules enforce standards.
MCP provides context.
Skills define the workflow.
The approach connects them.

That's the full system.
Here's how it connects.

Every piece. Three principles.

Perfect context
Language Servers, Rules, Monorepo

Always grounded
Context7, Firecrawl, Tool usage rules

Gated workflow
Superpowers, Two-layer verification, CI/CD

the approach

Gated Workflow

↑

Rules | **MCP Servers** | **Skills**

the tools

The Builder’s job isn’t writing code.

It’s building the machine that writes code well.